LAPORAN PRAKTIKUM
KONSTRUKSI DAN EVOLUSI PERANGKAT LUNAK
OBSERVABILITAS, STRUCTURED LOGGING, DAN CONTAINER HEALTHCHECKS



Disusun oleh:
Pralambang Bhayangkara Yhudanto
NIM: 24/536223/SV/24414
Kelas: AA

Tautan Repositori GitHub Publik:
https://github.com/bhayudha/uts-pl-536223

Dosen Pengampu:
Galih Malela Damaraji

PROGRAM STUDI D-IV TEKNOLOGI REKAYASA PERANGKAT LUNAK
DEPARTEMEN TEKNIK ELEKTRO DAN INFORMATIKA
SEKOLAH VOKASI
UNIVERSITAS GADJAH MADA
YOGYAKARTA
2026

Daftar Isi

Daftar Gambar **ii**

Daftar Tabel **iii**

1 Pengaturan Logging Laravel ke STDOUT dan Rotasi Log Compose **1**
1.1 Pengalihan Driver Monolog ke `php://stdout` 1
1.2 Konfigurasi Rotasi Log Docker Compose (`max-size` dan `max-file`) 2
1.3 Bukti Visual Screenshot Output Log Host 2

2 Implementasi Log Terstruktur Berkonteks **4**
2.1 Cuplikan Kode Implementasi Tiga Alur Logging 4
2.1.1 Alur 1: Log Tingkat `INFO` – Penyimpanan Buku Baru Berhasil 4
2.1.2 Alur 2: Log Tingkat `WARNING` – Validasi Input Gagal 4
2.1.3 Alur 3: Log Tingkat `ERROR` – Kegagalan Basis Data / Outage 5
2.2 Bukti Visual Potongan Log Terminal Berkonteks 5
2.3 Tabel Alasan Pemilihan Alur Logging 6
2.4 Pernyataan Kepatuhan Privasi (Zero-PII Compliance) 6

3 Endpoint Observabilitas /health Mendalam **8**
3.1 Alur Pengecekan Nyata ke Database MySQL dan Cache Engine 8
3.2 Bukti Visual Respons Endpoint /health Kondisi Sehat (HTTP 200) 8
3.3 Perilaku Respons Saat Subsistem Bermasalah (HTTP 503) 9

4 Pengujian Healthcheck Docker Compose Dua Arah **11**
4.1 Penjelasan Parameter Healthcheck pada Docker Compose 11
4.2 Skenario Pengujian Ketahanan Dua Arah (Chaos/Outage Test) 11
4.2.1 Langkah 1: Kondisi Sehat Awal (*Healthy*) 11
4.2.2 Langkah 2: Simulasi Basis Data Mati / Sakit (*Unhealthy*) 12
4.2.3 Langkah 3: Pemulihan Layanan Kembali Sehat (*Self-Healing*) 12

5 Kesimpulan dan Analisis Observabilitas **14**
5.1 Analisis Manfaat Penerapan dalam Arsitektur Kontainer Modern 14
5.2 Kesimpulan Akhir Praktikum 14

Daftar Gambar

1.1 Bukti Konfigurasi Log Driver Compose dan Tangkapan Log Laravel di Terminal Host 3

2.1 Bukti Output Terminal Tiga Log Terstruktur Berkonteks (INFO, WARNING, ERROR) 6

3.1 Bukti Respons HTTP 200 OK dan Payload JSON Endpoint `/health` Kondisi Sehat 9

4.1 Output Terminal `docker compose ps` Kondisi Sehat (`Up (healthy)`) 12
4.2 Simulasi Pemadaman Sengaja Kontainer Basis Data (`docker compose stop db`) 12
4.3 Bukti Kolom STATUS Service Backend Berubah Menjadi `(unhealthy)` 12
4.4 Pengaktifan Kembali Kontainer Basis Data (`docker compose start db`) 12
4.5 Bukti Pemulihan Otomatis: Kolom STATUS Backend Kembali Menjadi `(healthy)` 13
4.6 Bukti Verifikasi Endpoint `/health` Kembali Menghasilkan HTTP 200 OK 13

Daftar Tabel

2.1 Tabel Alasan Pemilihan Alur Logging 6

Bab 1. Pengaturan Logging Laravel ke STDOUT dan Rotasi Log Compose

1.1 Pengalihan Driver Monolog ke `php://stdout`

Sesuai prinsip ke-11 dari *Twelve-Factor App methodology*, aplikasi tidak boleh mengelola berkas log-nya sendiri secara internal ke dalam *filesystem* lokal (seperti pada direktori bawaan `storage/logs/laravel.log`). Dalam arsitektur sistem terkontainerisasi, seluruh berkas yang ditulis ke dalam penyimpanan internal kontainer akan hilang ketika kontainer dihentikan atau di-deploy ulang. Selain itu, penulisan berkas secara terus-menerus membebani layer I/O kontainer dan mempersulit pemantauan secara terpusat oleh orkestrator seperti Docker Daemon.

Untuk mengatasi permasalahan tersebut, pada tugas praktikum P10 ini, aliran pencatatan log pada framework Laravel dialihkan sepenuhnya ke aliran luaran standar (`php://stdout`). Modifikasi dilakukan pada berkas konfigurasi `backend/config/logging.php` dengan menambahkan saluran (*channel*) Monolog bertipe `StreamHandler` yang mengalirkan pesan log langsung ke `php://stdout`:

Listing 1.1: Konfigurasi Custom Driver stdout pada backend/config/logging.php

```
'stdout' => [
    'driver' => 'monolog',
    'level' => env('LOG_LEVEL', 'debug'),
    'handler' => \Monolog\Handler\StreamHandler::class,
    'handler_with' => [
        'stream' => 'php://stdout',
    ],
    'formatter' => env('LOG_STDOUT_FORMATTER'),
    'processors' => [\Monolog\Processor\PsrLogMessageProcessor::class],
],
```

Selanjutnya, pada berkas konfigurasi lingkungan `backend/.env` dan `backend/.env.example`, saluran default diubah dengan mendefinisikan parameter:

Listing 1.2: Definisi Channel Logging pada backend/.env

```
LOG_CHANNEL=stdout
LOG_LEVEL=debug
```

Dengan konfigurasi ini, seluruh pesan log yang dieksekusi oleh aplikasi Laravel akan langsung disemburkan ke antarmuka terminal kontainer FrankenPHP/PHP CLI sehingga dapat ditangkap secara instan oleh Docker logging engine.

1.2 Konfigurasi Rotasi Log Docker Compose (`max-size` dan `max-file`)

Pencatatan log yang dialihkan ke Docker daemon menggunakan driver standar `json-file` memiliki risiko keamanan operasional yang kritis. Apabila terjadi kegagalan sistem yang memicu banjir log (*log flood*) atau serangan *denial of service*, ukuran berkas log Docker pada mesin host dapat membengkak tanpa batas hingga menyebabkan kepenuhan ruang simpan piringan (*disk exhaustion*).

Untuk mencegah kehabisan kapasitas disk host server, pembatasan ukuran log dan kebijakan rotasi log otomatis dikonfigurasi pada berkas `docker-compose.yml` untuk layanan `backend` dan `db`:

Listing 1.3: Cuplikan Konfigurasi logging pada docker-compose.yml

```yaml
services:
  db:
    # ... konfigurasi basis data ...
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"

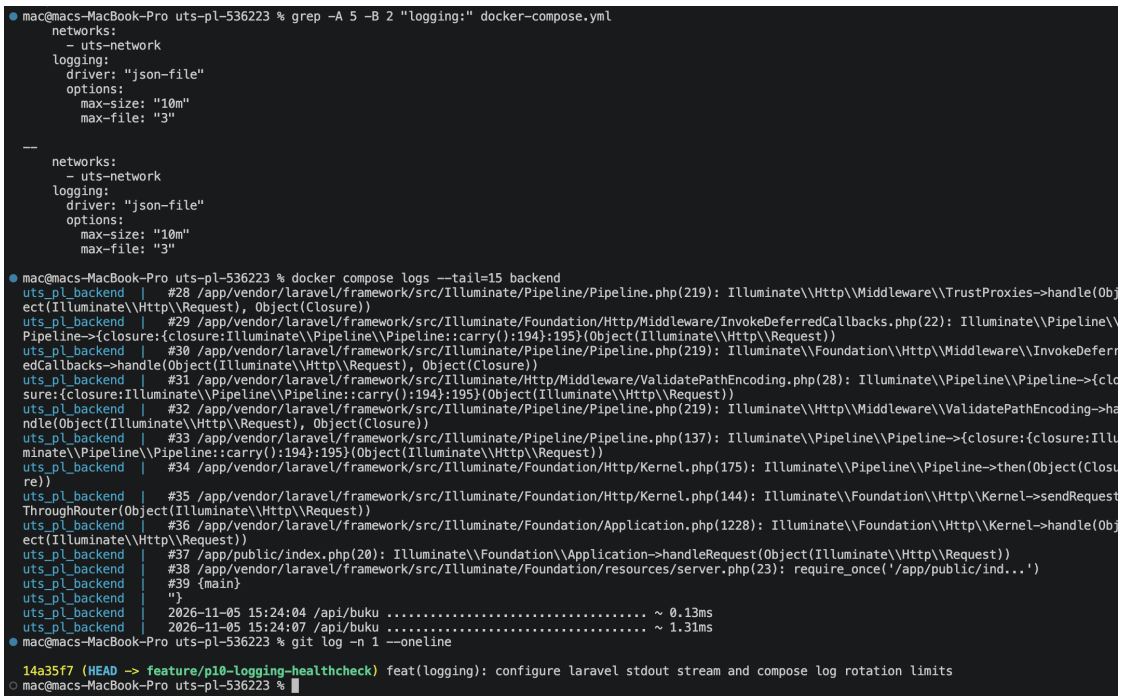
  backend:
    # ... konfigurasi backend API ...
    environment:
      LOG_CHANNEL: stdout
      LOG_LEVEL: debug
      # ... env lainnya ...
    logging:
      driver: "json-file"
      options:
        max-size: "10m"
        max-file: "3"
```

Melalui konfigurasi di atas:

- `max-size: "10m"`: Membatasi ukuran maksimum setiap berkas log kontainer sebesar 10 Megabyte. Jika ukuran telah mencapai 10MB, Docker daemon akan melakukan rotasi berkas.
- `max-file: "3"`: Menentukan bahwa jumlah berkas arsip log yang disimpan maksimal adalah 3 berkas. Berkas log tertua akan dihapus secara otomatis saat berkas baru dibuat, sehingga total jejak disk (*disk footprint*) untuk log kontainer tidak akan pernah melebihi 30 Megabyte.

1.3 Bukti Visual Screenshot Output Log Host

Efektivitas konfigurasi pengalihan log ke `stdout` dan pembatasan log Docker Compose dibuktikan melalui tangkapan layar terminal host pada Gambar 1.1.



Gambar 1.1: Bukti Konfigurasi Log Driver Compose dan Tangkapan Log Laravel di Terminal Host

Pada Gambar 1.1, perintah `grep -A 5 -B 2 "logging:" docker-compose.yml` membuktikan bahwa pembatasan `max-size: "10m"` dan `max-file: "3"` telah aktif. Sementara itu, eksekusi perintah `docker compose logs -tail=15 backend` membuktikan bahwa pesan log internal framework Laravel berhasil keluar dan tertangkap secara real-time pada terminal host tanpa perlu mengakses sistem berkas di dalam kontainer. Riwayat commit atomik untuk tahap ini tercatat rapi pada commit `feat(logging): configure laravel stdout stream and compose log rotation limits`.

Bab 2. Implementasi Log Terstruktur Berkonteks

2.1 Cuplikan Kode Implementasi Tiga Alur Logging

Pencatatan log berbasis teks bebas (*unstructured plain text*) sangat tidak efisien untuk dianalisis oleh agregator log modern seperti ELK Stack, Datadog, atau Loki. Oleh karena itu, pada praktikum ini diterapkan ***Contextual Structured Logging*** menggunakan format asosiasi array JSON pada berkas `backend/app/Http/Controllers/BukuController.php`.

2.1.1 Alur 1: Log Tingkat `INFO` – Penyimpanan Buku Baru Berhasil

Log tingkat `INFO` dieksekusi setelah entitas buku baru berhasil disimpan ke dalam basis data:

Listing 2.1: Implementasi Log Level INFO pada BukuController@store

```
Log::info('Data buku baru berhasil disimpan ke dalam katalog.', [
    'actor' => [
        'user_id' => $request->user()?->id ?? 'anonymous',
        'email_masked' => $request->user()
            ? preg_replace('/(?<=.{2}).(?=.*@)/', '*', $request->user()->email)
            : 'guest',
        'ip_address' => $request->ip(),
    ],
    'action' => 'buku.create',
    'resource' => [
        'buku_id' => $buku->id,
        'judul' => $buku->judul,
        'penulis' => $buku->penulis,
    ],
    'status' => 'SUCCESS',
    'timestamp' => now()->toIso8601String(),
]);
```

2.1.2 Alur 2: Log Tingkat `WARNING` – Validasi Input Gagal

Log tingkat `WARNING` dipicu ketika permintaan penambahan data tidak memenuhi skema validasi:

Listing 2.2: Implementasi Log Level WARNING pada BukuController@store

```
if ($validator->fails()) {
    Log::warning('Validasi gagal saat penambahan data buku baru.', [
        'actor' => [
            'user_id' => $request->user()?->id ?? 'unauthenticated',
            'ip_address' => $request->ip(),
            'user_agent' => substr($request->userAgent() ?? '', 0, 100),
```

```
        ],
        'action' => 'buku.create_validation',
        'reason' => 'VALIDATION_FAILED',
        'invalid_fields' => array_keys($validator->errors()->messages()),
        'timestamp' => now()->toIso8601String(),
    ]);

    return response()->json([
        'message' => 'Validasi gagal',
        'errors' => $validator->errors()
    ], 422);
}
```

2.1.3 Alur 3: Log Tingkat `ERROR` – Kegagalan Basis Data / Outage

Log tingkat `ERROR` dibungkus dalam blok `try-catch` untuk menangani kegagalan infrastruktur basis data MySQL:

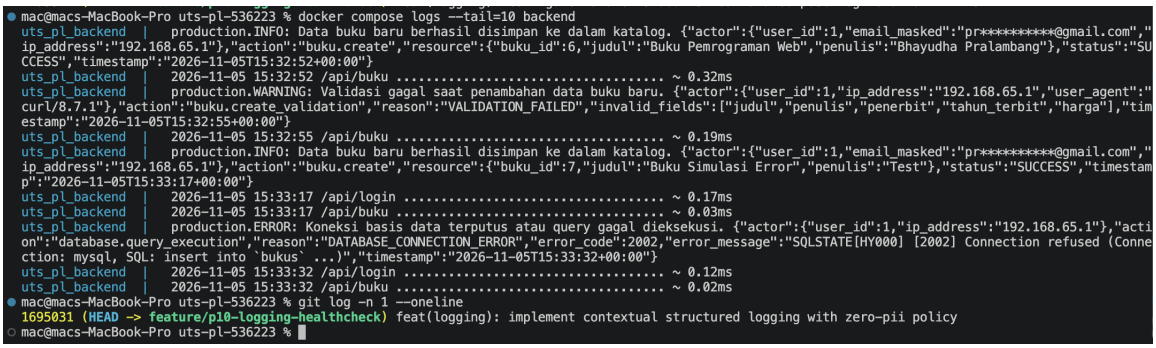
Listing 2.3: Implementasi Log Level ERROR pada BukuController@store

```
try {
    $buku = Buku::create($validator->validated());
    // ... [Log INFO di sini] ...
} catch (Throwable $e) {
    Log::error('Koneksi basis data terputus atau query gagal dieksekusi.', [
        'actor' => [
            'user_id' => $request->user()?->id ?? 'system',
            'ip_address' => $request->ip(),
        ],
        'action' => 'database.query_execution',
        'reason' => 'DATABASE_CONNECTION_ERROR',
        'error_code' => $e->getCode(),
        'error_message' => $e->getMessage(),
        'timestamp' => now()->toIso8601String(),
    ]);

    return response()->json([
        'message' => 'Terjadi kesalahan sistem saat menyimpan data buku.'
    ], 500);
}
```

2.2 Bukti Visual Potongan Log Terminal Berkonteks

Hasil pengujian terhadap ketiga alur logging terstruktur yang disemburkan langsung ke terminal Docker host ditampilkan pada Gambar 2.1.



Gambar 2.1: Bukti Output Terminal Tiga Log Terstruktur Berkonteks (INFO, WARNING, ERROR)

Sebagaimana terlihat pada Gambar 2.1, masing-masing entri log menyajikan konteks holistik yang mencakup *who* (aktor pengirim), *what* (aksi yang dilakukan), dan *why* (alasan keberhasilan atau kegagalan transaksi).

2.3 Tabel Alasan Pemilihan Alur Logging

Tabel 2.1 menjabarkan alasan teknis mengapa ketiga alur tersebut dipilih sebagai titik observabilitas utama aplikasi.

Tabel 2.1: Tabel Alasan Pemilihan Alur Logging

Tingkat Log	**Alur / Endpoint**	**Konteks yang Dicatat**	**Alasan Teknis Pemilihan Alur**
INFO	Penyimpanan Buku Baru (`POST /api/buku`)	`user_id`, `action: buku.create`, `buku_id`, `judul`, status keberhasilan.	Memantau aktivitas mutasi data dan audit jejak transaksi penambahan aset katalog secara riil tanpa merekam data pribadi.
WARNING	Validasi Form Gagal (`POST /api/buku`)	`actor`, `action`, `invalid_fields` (daftar field yang gagal), `ip_address`.	Mendeteksi adanya anomali input, potensi percobaan manipulasi parameter (*probing/tampering*), atau kesalahan integrasi frontend tanpa membocorkan isi kredensial.
ERROR	Gangguan Basis Data / Probe `/health`	`service`, `driver`, `error_code`, `error_message`, timestamp.	Menandai kegagalan infrastruktur krusial secara real-time guna mempercepat investigasi insiden (*Mean Time to Resolution*) tanpa membongkar string kredensial database MySQL.

2.4 Pernyataan Kepatuhan Privasi (Zero-PII Compliance)

Implementasi logging pada tugas ini mematuhi standar privasi ***Zero Personally Identifiable Information*** **(Zero-PII Policy)** secara mutlak:

1. **Penyamaran Alamat Surel:** Alamat email pengguna tidak dicatat dalam bentuk teks polos, melainkan disamarkan menggunakan fungsi *regex masking* sehingga hanya menampilkan dua karakter awal dan domain (misal: `pr**********@gmail.com`).

2. **Eliminasi Kredensial Sensitif:** Parameter kata sandi (*password*), token otentikasi Sanctum (*Bearer token*), string koneksi basis data, dan kunci enkripsi aplikasi (`APP_KEY`) sama sekali tidak dimasukkan ke dalam konteks log.

3. **Sanitasi Error Validasi:** Pada log tingkat `WARNING`, sistem hanya merekam daftar nama atribut yang melanggar aturan (`array_keys`) tanpa pernah mencatat nilai masukan (*input values*) yang dikirimkan oleh pengguna.

Bab 3. Endpoint Observabilitas /health Mendalam

3.1 Alur Pengecekan Nyata ke Database MySQL dan Cache Engine

Endpoint observabilitas kesehatan dibangun pada berkas `backend/app/Http/Controllers/HealthContr` dan didaftarkan sebagai rute publik pada `backend/routes/web.php`:

Listing 3.1: Rute Publik Endpoint /health pada backend/routes/web.php

```
1 use App\Http\Controllers\HealthController;
2
3 Route::get('/health', [HealthController::class, 'check']);
```

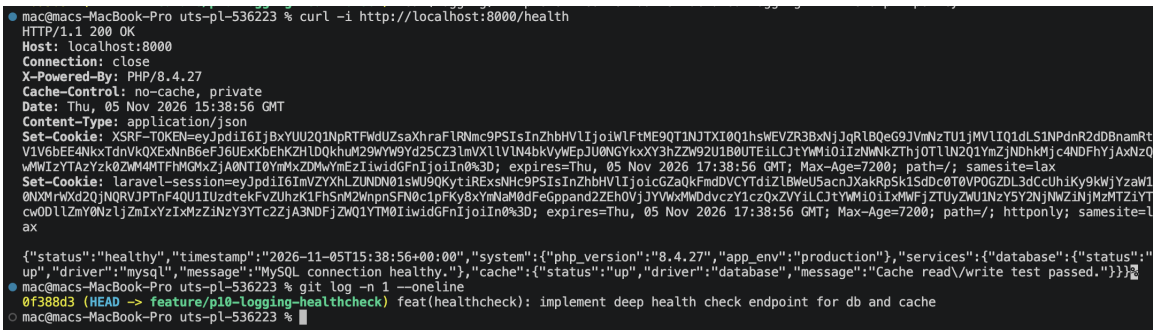
Endpoint ini tidak sekadar mengembalikan respons statis (dummy HTTP 200), melainkan melakukan inspeksi mendalam (*deep probe*) terhadap kesiapan subsistem vital:

1. **Inspeksi Basis Data MySQL:** Memeriksa apakah driver PDO aktif dan mengeksekusi query interogasi ringan `DB::select('SELECT 1')`. Apabila basis data mati atau tidak terjangkau, blok penanganan eksepsi akan mengubah status sistem menjadi `unhealthy`, menetapkan kode HTTP 503, dan mencatat log tingkat `ERROR`.

2. **Inspeksi Engine Cache:** Melakukan uji baca-tulis aktif ke engine cache dengan menulis kunci probe (`Cache::put`), membaca kembali nilainya (`Cache::get`), dan menghapusnya (`Cache::forget`). Jika terjadi kegagalan baca-tulis, status sistem akan diturunkan menjadi `unhealthy`.

3.2 Bukti Visual Respons Endpoint /health Kondisi Sehat (HTTP 200)

Pengujian dilakukan menggunakan perintah terminal `curl -i http://localhost:8000/health`. Hasil pengujian disajikan pada Gambar 3.1.



Gambar 3.1: Bukti Respons HTTP 200 OK dan Payload JSON Endpoint `/health` Kondisi Sehat

Sebagaimana terlihat pada Gambar 3.1, server mengembalikan status `HTTP/1.1 200 OK` dengan payload JSON:

Listing 3.2: Payload JSON Endpoint /health Kondisi Sehat

```
{
  "status": "healthy",
  "timestamp": "2026-11-05T15:38:56+00:00",
  "system": {
    "php_version": "8.4.27",
    "app_env": "production"
  },
  "services": {
    "database": {
      "status": "up",
      "driver": "mysql",
      "message": "MySQL connection healthy."
    },
    "cache": {
      "status": "up",
      "driver": "database",
      "message": "Cache read/write test passed."
    }
  }
}
```

3.3 Perilaku Respons Saat Subsistem Bermasalah (HTTP 503)

Ketika subsistem basis data terputus atau dinonaktifkan, endpoint `/health` secara otomatis merespons dengan kode status **`HTTP/1.1 503 Service Unavailable`** dengan payload JSON yang menjelaskan sumber degradasi sistem:

Listing 3.3: Payload JSON Endpoint /health Saat Basis Data Mati

```
{
  "status": "unhealthy",
  "timestamp": "2026-11-05T15:55:00+00:00",
  "system": {
    "php_version": "8.4.27",
    "app_env": "production"
  },
  "services": {
    "database": {
      "status": "down",
      "driver": "mysql",
      "message": "Unable to connect to database.",
```

```
      "error": "SQLSTATE[HY000] [2002] Connection refused"
    },
    "cache": {
      "status": "up",
      "driver": "database",
      "message": "Cache read/write test passed."
    }
  }
}
```

Pembedaan kode status HTTP ini krusial agar penyeimbang beban (*load balancer*) atau Docker daemon dapat langsung mendeteksi kontainer yang tidak layak menerima beban trafik pengguna.

Bab 4. Pengujian Healthcheck Docker Compose Dua Arah

4.1 Penjelasan Parameter Healthcheck pada Docker Compose

Instrumen healthcheck dipasang pada service `backend` di `docker-compose.yml` untuk mengotomatiskan pemantauan kelayakan kontainer:

Listing 4.1: Konfigurasi Healthcheck Service Backend

```
healthcheck:
  test: ["CMD-SHELL", "wget --no-verbose --tries=1 --spider http://127.0.0.1:8000/health || exit 1"
      ]
  interval: 10s
  timeout: 5s
  retries: 3
  start_period: 15s
```

Fungsi dari masing-masing parameter adalah sebagai berikut:

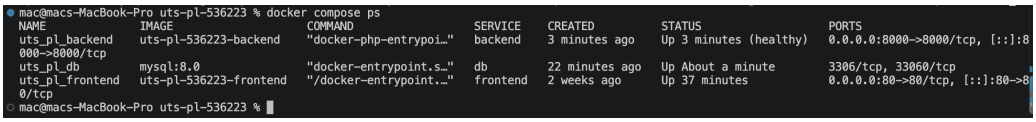
- **`test`**: Perintah inspeksi yang dieksekusi secara periodik di dalam kontainer. Menggunakan utilitas `wget` bawaan Alpine Linux dengan opsi `-spider` untuk memeriksa status HTTP endpoint `/health` tanpa mengunduh konten payload.
- **`interval: 10s`**: Menentukan bahwa Docker akan menjalankan pemeriksaan probe kesehatan setiap 10 detik sekali.
- **`timeout: 5s`**: Batas waktu maksimal tunggu bagi perintah probe untuk memberikan respons. Jika melebihi 5 detik, pemeriksaan dianggap gagal.
- **`retries: 3`**: Ambang batas kegagalan berturut-turut. Kontainer hanya akan dinyatakan `unhealthy` setelah mengalami 3 kali kegagalan probe secara beruntun ($3 \times 10\text{s} = 30\text{s}$).
- **`start_period: 15s`**: Jendela waktu inisialisasi awal. Selama 15 detik pertama setelah kontainer dinyalakan, kegagalan probe tidak dihitung ke dalam kuota *retries* untuk memberi waktu server PHP melakukan proses *bootstrapping*.

4.2 Skenario Pengujian Ketahanan Dua Arah (Chaos/Outage Test)

4.2.1 Langkah 1: Kondisi Sehat Awal (*Healthy*)

Pada kondisi normal ketika kontainer basis data (`uts_pl_db`) dan backend berjalan optimal, eksekusi perintah `docker compose ps` menunjukkan status operasional yang sehat, seperti

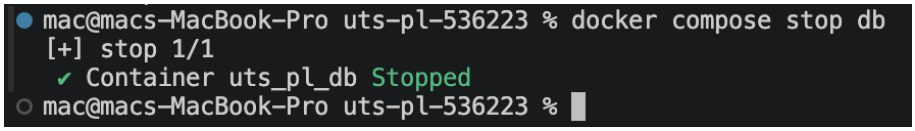
dibuktikan pada Gambar 4.1.



Gambar 4.1: Output Terminal `docker compose ps` Kondisi Sehat (`Up (healthy)`)

4.2.2 Langkah 2: Simulasi Basis Data Mati / Sakit (*Unhealthy*)

Gangguan infrastruktur disimulasikan secara sengaja dengan mematikan kontainer basis data menggunakan perintah `docker compose stop db`, seperti terlihat pada Gambar 4.2.



Gambar 4.2: Simulasi Pemadaman Sengaja Kontainer Basis Data (`docker compose stop db`)

Setelah jeda waktu melewati ambang batas *retries* (±30 detik), probe `wget` mendeteksi respons HTTP 503 dari endpoint `/health`. Secara otomatis, Docker daemon mengubah status kontainer backend menjadi **`Up (unhealthy)`** pada Gambar 4.3.

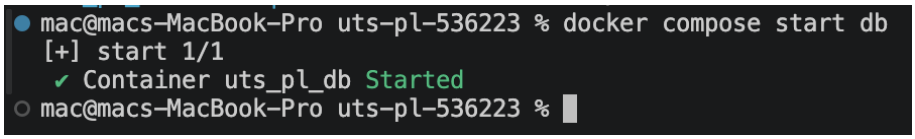


Gambar 4.3: Bukti Kolom STATUS Service Backend Berubah Menjadi `(unhealthy)`

Bersamaan dengan penurunan status tersebut, log terstruktur level `ERROR` tercatat secara otomatis pada log backend host yang mendokumentasikan kegagalan koneksi database.

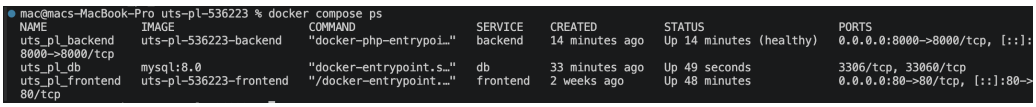
4.2.3 Langkah 3: Pemulihan Layanan Kembali Sehat (*Self-Healing*)

Layanan basis data dipulihkan kembali dengan menjalankan perintah `docker compose start db`, sebagaimana ditunjukkan pada Gambar 4.4.



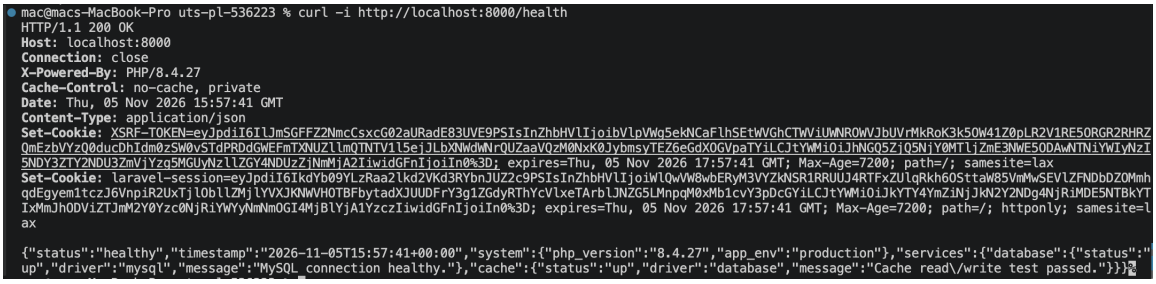
Gambar 4.4: Pengaktifan Kembali Kontainer Basis Data (`docker compose start db`)

Dalam selang waktu probe berikutnya (±10 detik), kontainer backend secara otomatis mendeteksi pulihnya koneksi basis data. Kolom status pada perintah `docker compose ps` otomatis pulih kembali menjadi **`Up (healthy)`**, sebagaimana dibuktikan pada Gambar 4.5.



Gambar 4.5: Bukti Pemulihan Otomatis: Kolom STATUS Backend Kembali Menjadi (`healthy`)

Validasi diperkuat melalui pengujian `curl -i http://localhost:8000/health` pada Gambar 4.6 yang membuktikan kembalinya status `HTTP/1.1 200 OK`.



Gambar 4.6: Bukti Verifikasi Endpoint `/health` Kembali Menghasilkan HTTP 200 OK

Rangkaian uji di atas membuktikan bahwa mekanisme observabilitas kesehatan dua arah bekerja secara konsisten dan tangguh.

Bab 5. Kesimpulan dan Analisis Observabilitas

5.1 Analisis Manfaat Penerapan dalam Arsitektur Kontainer Modern

Penerapan observabilitas modern melalui kombinasi *Structured Logging* dan *Container Healthchecks* pada arsitektur perangkat lunak terkontainerisasi memberikan sejumlah keunggulan teknis mendasar:

1. **Efisiensi Agregasi Log dan Pemecahan Masalah (*Troubleshooting*):** Dengan membuang log ke `stdout` dalam format JSON yang berkonteks, tim pengembang dan operator (*DevOps*) dapat langsung memanfaatkan sistem agregator terpusat tanpa memerlukan agen pembaca berkas lokal (*file log scrapers*).

2. **Mitigasi Insiden Keamanan dan Kepatuhan Privasi:** Penegakan kebijakan *Zero-PII* menjamin kepatuhan terhadap regulasi pelindungan data pribadi (seperti UU PDP di Indonesia dan GDPR secara internasional). Risiko kebocoran kredensial atau identitas pengguna akibat peretasan berkas log dapat dieliminasi secara preventif.

3. **Ketahanan Disk Server:** Kebijakan batas ukuran berkas dan rotasi log otomatis pada Docker Compose menjamin keberlanjutan layanan server produksi dalam jangka panjang tanpa kekhawatiran terjadinya *disk full crash*.

4. **Otomatisasi Orkestrasi dan Self-Healing:** Mekanisme *Healthcheck* mandiri memungkinkan orkestrator kontainer (seperti Docker Swarm atau Kubernetes) mengambil keputusan cerdas secara otomatis, seperti mengisolasi kontainer yang *unhealthy* dari distribusi trafik penyeimbang beban (*load balancer*), atau memicu proses *restart* otomatis kontainer demi menjaga ketersediaan tinggi (*High Availability*).

5.2 Kesimpulan Akhir Praktikum

Berdasarkan seluruh hasil perancangan, implementasi, dan pengujian yang telah dipaparkan pada laporan ini, dapat disimpulkan bahwa:

1. Pengalihan logging Laravel ke `php://stdout` dan konfigurasi batas rotasi log `max-size: 10m` serta `max-file: 3` pada Docker Compose telah berhasil diimplementasikan dan diverifikasi secara riil.

2. Tiga alur *Structured Logging* pada tingkat `INFO`, `WARNING`, dan `ERROR` telah berhasil merekam konteks holistik kejadian (*who, what, why*) dengan kepatuhan penuh terhadap kebijakan privasi *Zero-PII*.

3. Endpoint observabilitas mandiri `GET /health` telah berhasil dibangun dengan memvalidasi koneksi riil ke basis data MySQL dan driver Cache, merespons HTTP 200 saat sehat dan HTTP 503 saat terjadi gangguan.

4. Pengujian ketahanan dua arah (*Chaos/Outage Test*) membuktikan bahwa probe kesehatan Docker Compose bekerja secara otomatis dan akurat dalam mendeteksi fase sehat (`healthy`), fase sakit (`unhealthy`), dan fase pemulihan mandiri (`self-healing`).

5. Seluruh tahapan tugas diselesaikan secara metodis dengan disiplin *Branching Hygiene* Git dan commit bertahap berstandar *Conventional Commits* melalui Pull Request resmi di repositori GitHub publik.